

ETL-SQL Grammar & Orchestration Syntax

This document is the authoritative reference for the ETL-SQL scripting language. It defines every statement type, clause, and keyword — everything needed to write, administer, and automate with ETL-SQL.

Sections marked [PROPOSED] contain syntax designed during the Report Portal planning phase that has not yet been implemented. All other sections reflect the current engine.

1. Variable & State Management

1.1 DECLARE

Defines one or more variables. The data type is optional; omitting it defaults to **ANY** (inferred from the assigned value).

```
DECLARE @name      STRING      = 'Chuck';
DECLARE @note      MARKDOWN    = '# Hello';
DECLARE @id        INT         = 101;
DECLARE @rate      DECIMAL(10,4) = 1.2345;
DECLARE @icon      IMAGE       = 'C:\Data\icon.png';
DECLARE @range     MINMAX(INT)  = (1, 100);
DECLARE @files     LIST(PATH)   = ('C:\Data\1.csv', 'C:\Data\2.csv');
DECLARE @path      PATH        = 'C:\Data\1.csv';

-- Multiple variables in one statement
DECLARE @list LIST = (1, 2, 3), @count INT = 0;

-- No type – inferred at runtime
DECLARE @value = 'hello';

-- Table variable – acts as an in-memory temp table
DECLARE @inventory TABLE;
```

Supported Data Types:

Category	Types
Numeric	INT, INTEGER, BIGINT, SMALLINT, TINYINT, BIT, BOOLEAN, BOOL, DECIMAL, NUMERIC, MONEY, SMALLMONEY, FLOAT, REAL, DOUBLE
Temporal	DATE, TIME, DATETIME, DATETIME2, SMALLDATETIME, DATETIMEOFFSET, TIMESTAMP
Character	CHAR, VARCHAR, VARCHAR2, NCHAR, NVARCHAR, TEXT, NTEXT, STRING, MARKDOWN
Binary	BINARY, VARBINARY, IMAGE, BLOB, LOB
Specialized	XML, JSON, UNIQUEIDENTIFIER, UUID, GUID, GEOMETRY, GEOGRAPHY, HIERARCHYID, VARIANT, SQL_VARIANT, ANY, PATH, MINMAX, CURSOR, ENCRYPTED, VECTOR, SENSITIVE, SECRET

Category Types

Collections `LIST`, `LIST(<type>)`, `TABLE`

1.2 Specialty Types

Specialty types carry semantic meaning beyond a plain string or number. They influence how values are stored, validated, masked, or rendered. The table below gives a quick summary; detailed notes follow.

Type	Stored as	When behavior activates
<code>PATH</code>	String	At file I/O — normalizes separators, resolves relative paths, validates security boundaries
<code>JSON</code>	String	At assignment — validates well-formedness immediately; enables <code>JSON_VALUE</code> , <code>JSON_QUERY</code> , etc.
<code>XML</code>	String	At assignment — validates well-formedness immediately; enables <code>XMLVALUE</code> , <code>XMLQUERY</code> , etc.
<code>MARKDOWN</code>	String	At render time — Report Portal renders as rich text; CLI treats as plain string
<code>LIST</code>	Collection	At iteration or index access — can be strictly typed, e.g. <code>LIST(INT)</code>
<code>MINMAX</code>	Struct	At declaration — gives a <code>.MIN</code> and <code>.MAX</code> member; inner type annotation is documentary
<code>ENCRYPTED</code>	String	At runtime — masked in <code>SHOW VARIABLES</code> ; auto-decrypts <code>ENC:</code> values when assigned to non-SENSITIVE targets or passed to secure parameters
<code>SENSITIVE</code>	Any	At runtime — masked in <code>SHOW VARIABLES</code> ; auto-decrypts <code>ENC:</code> values when assigned to non-SENSITIVE targets or passed to secure parameters
<code>SECRET</code>	Any	At session end — nullified in memory automatically; same masking/auto-decryption as <code>SENSITIVE</code>

`PATH`

Stored as a string at declaration time. When the value is passed to any file I/O operation, `ResolvePath()` activates and:

- Strips surrounding double-quotes that Windows *Copy as path* adds (e.g. `"C:\tmp\file.csv"`)
- Accepts a connector name as the root segment: `MyDrive/subdir/file.csv` where `MyDrive` is a configured file or SFTP connector
- Resolves relative paths against the script's working directory
- Validates the resolved path against configured security boundaries (allowed root paths, permitted file extensions)

```
DECLARE @out PATH = 'C:\Data\results.csv';           -- absolute
DECLARE @src PATH = 'SftpServer/inbox/feed.csv';     -- connector-relative
```

Use **PATH** instead of **STRING** whenever a variable holds a filesystem path. This makes intent explicit to the LSP, agents, and the security layer.

JSON

Validated at assignment — an invalid JSON string raises an **ExecutionException** immediately at the **DECLARE** or **SET** line. Stored as a string internally after passing validation.

Declaring a variable as **JSON** unlocks the full JSON function set on that value:

Function	Purpose
JSON_VALUE(@v, '\$.path')	Extract a scalar value at the given JSONPath
JSON_QUERY(@v, '\$.path')	Extract an object or array fragment
JSON_MODIFY(@v, '\$.path', val)	Return a copy with a value updated or inserted
ISJSON(@v)	Returns 1 if the string is valid JSON, 0 otherwise
JSON_EXISTS(@v, '\$.path')	Returns 1 if the path exists
JSON_TABLE(@v, '\$.path')	Expand a JSON array into a table
OPENJSON(@v)	SQL-Server-style JSON rowset expansion

```
DECLARE @payload JSON = '{"order":{"id":42,"total":99.95}}';
SELECT JSON_VALUE(@payload, '$.order.id') AS id,
       JSON_VALUE(@payload, '$.order.total') AS total;
```

XML

Validated at assignment — an invalid XML string raises an **ExecutionException** immediately at the **DECLARE** or **SET** line. Stored as a string internally after passing validation.

Function	Purpose
XMLVALUE(@v, xpath)	Extract a scalar value using XPath
XMLQUERY(@v, xpath)	Extract an XML fragment
XMLTABLE(@v, xpath)	Expand an XML document into a table
XMLEXISTS(@v, xpath)	Returns 1 if the XPath matches any node
XMLELEMENT(name, content)	Construct an XML element

```
DECLARE @doc XML = '<root><item id="1">Alpha</item></root>';
SELECT XMLVALUE(@doc, '//item[@id=1]') AS name;
```

MARKDOWN

Stored as a plain string. No validation is performed at assignment (any string is technically valid markdown). In script execution (CLI, headless), it is treated identically to **STRING**. In the **Report Portal**, a **MARKDOWN** variable bound to a visual component is rendered as HTML-formatted rich text — headers, bold, lists, tables, and code blocks are all interpreted.

```
DECLARE @summary MARKDOWN = '## Run Complete\n- Records: 1000\n- Errors: 0';
```

Use **MARKDOWN** as a rendering hint for the Report Portal. It has no runtime overhead and signals intent to both the engine and AI agents building dashboards.

LIST

An ordered, index-accessible collection. Elements can be iterated with **FOREACH** or accessed by position. Optionally strongly typed:

```
DECLARE @ids    LIST(INT)  = (1, 2, 3);
DECLARE @paths  LIST(PATH) = ('C:\a.csv', 'C:\b.csv');
DECLARE @mixed  LIST      = ('hello', 42, NULL); -- inferred
```

The inner type annotation (e.g. **LIST(INT)**) is enforced as a cast — assigning a non-castable value raises an error. **LIST** without a type accepts any value.

MINMAX

A structured two-value type with **.MIN** and **.MAX** members. It is the only non-collection specialty type that provides member access via dot notation.

```
DECLARE @range MINMAX(INT) = (1, 100);

PRINT @range.MIN; -- 1
PRINT @range.MAX; -- 100

SET @range.MIN = 10;
SET @range.MAX = 90;
```

The inner type annotation (**MINMAX(INT)**, **MINMAX(DECIMAL)**, etc.) is documentary — the engine stores **.MIN** and **.MAX** as generic **object** values and does not enforce the inner type at runtime. Use it to communicate intent.

Common uses: date window boundaries, numeric filter ranges, batch size limits.

ENCRYPTED

The canonical type for variables that hold an `ENC:...` value — a ciphertext string produced by `ENCRYPT()` or stored in the credentials vault. This type ensures the value is protected throughout its lifecycle in the engine.

Engine Behaviors:

- **Runtime Masking:** The variable is marked as sensitive. `SHOW VARIABLES` and `PRINT` output will mask the value (displaying `*****` or `ENC:*****`) unless `SET SHOW_PASSWORD ON` is active.
- **Auto-Decryption:** The engine automatically decrypts the `ENC:` value in two scenarios:
 1. When passed to a secure connector parameter (`PASSWORD`, `API_KEY`, `SSH_KEY_PAIR.PASSPHRASE`, etc.).
 2. When evaluated in an expression that is assigned to a non-SENSITIVE target or used in a comparison.
- **Lint Protection (SEC-4):** The linter flags any attempt to concatenate or pass an `ENCRYPTED` variable to insecure sinks (like `SEND EMAIL` bodies or file writes).

```
USE PASSWORD = 'my-master-key';
DECLARE @pwd ENCRYPTED = 'ENC:abc123==';

-- Connection automatically handles decryption
CREATE CONNECTION MyDb ON MSSQL(PASSWORD = @pwd);

-- Linter will warn here, and runtime will mask output
PRINT 'The password is: ' + @pwd;
```

SENSITIVE

Sets the `IsSensitive` runtime flag on the variable. Three effects activate immediately:

1. **SHOW VARIABLES masking** — the value is replaced with `*****` in all variable listing output (unless `SET SHOW_PASSWORD ON` is active).
2. **ENC: auto-decryption** — if the value begins with `ENC:`, the engine automatically decrypts it when the variable is passed to a secure connector parameter (`PASSWORD`, `API_KEY`, `SSH_KEY_PAIR.PASSPHRASE`, etc.). This requires `USE SCRIPT PASSWORD` or a master password to be set.
3. **Lint taint tracking** — if you assign a `SENSITIVE` variable into a new variable (`SET @other = @pwd`), the linter marks `@other` as sensitive too, propagating SEC-4 warnings forward.

`SENSITIVE` does **not** prevent you from printing the value — `PRINT @sensitiveVar` will output the plaintext. The SEC-4 lint rule warns you when you do this, but the runtime does not block it.

```
DECLARE @dbPass SENSITIVE = 'ENC:abc123=='; -- masked in SHOW VARIABLES,
decrypted at connect time
USE SCRIPT PASSWORD 'my-master-key';
```

```
OPEN CONNECTION MyDb WITH (PASSWORD = @dbPass); -- @dbPass decrypted here
automatically
```

SECRET

Implicitly sets `IsSensitive = true`, providing the same masking and auto-decryption effects as `SENSITIVE`. However, it introduces a strict **Zero-Trust memory policy**:

- **Session-End Purge:** To minimize exposure of ultra-sensitive data (like MFA tokens or temporary access keys), variables declared as `SECRET` are explicitly nullified in **all** memory scopes (global and nested) immediately after the evaluator finishes execution, regardless of whether the script succeeded or failed.
- **Transience Hint:** Signals to the engine and auditors that this value is a short-lived token that should never be persisted or logged.

```
DECLARE @dbPassword SENSITIVE = 'ENC:abc123=='; -- persistent for session
DECLARE @bearerToken SECRET = GetBearerToken(...); -- purged from RAM on
finish
```

[!NOTE] Log scrubbing is always active regardless of variable type. The engine automatically redacts patterns like `password=value`, `token=value`, and any `ENC:...` constant found in log messages or connection string text. Variables are scrubbed by pattern, not by metadata — so `SENSITIVE/SECRET` masking applies specifically to `SHOW VARIABLES` output and the auto-decrypt pathway.

1.3 SET

Assigns a new value to an existing variable.

```
SET @name = 'Charles';
SET @count = @count + 1;
SET @label = UPPER(@name) + '_PROCESSED';

-- Member access assignment
SET @range.MIN = 5;
SET @range.MAX = 50;
```

1.4 System Variables

Read-only variables that track session state and performance. These are automatically updated by the engine.

Variable	Description	Example Usage
@@ROWCOUNT	Rows affected by the last DML or returned by the last <code>SELECT</code> .	<code>IF @@ROWCOUNT = 0 PRINT 'No data found';</code>

Variable	Description	Example Usage
@@ERROR	Integer error code of the preceding statement (0 = success).	IF @@ERROR <> 0 RETURN;
@@VERSION	Full engine version and build metadata string.	PRINT @@VERSION;
@@TRANCOUNT	Current transaction nesting level (0 = no active transaction).	IF @@TRANCOUNT > 0 COMMIT;
@@FETCH_STATUS	Cursor/Foreach fetch status. 0 = Success, -1 = End of list.	WHILE @@FETCH_STATUS = 0 BEGIN ... END
@@TOTAL_SPILLED_BYTES	Cumulative bytes written to disk for all spill operations this session.	IF @@TOTAL_SPILLED_BYTES > 1073741824 PRINT 'High disk pressure';
@@LAST_EXEC_MS	Milliseconds taken by the last statement.	IF @@LAST_EXEC_MS > 5000 PRINT 'Warning: slow query';
@@PEAK_MEMORY_MB	Peak working-set memory in MB for the current process.	PRINT 'Peak RAM: ' + @@PEAK_MEMORY_MB + ' MB';
@@SUBQUERY_CACHE_HITS	Scalar subquery results retrieved from session cache.	PRINT 'Cache Hits: ' + @@SUBQUERY_CACHE_HITS;
@@SUBQUERY_CACHE_MISSES	Scalar subquery evaluations that required execution.	PRINT 'Cache Misses: ' + @@SUBQUERY_CACHE_MISSES;
@@SORT_SPILLS	External sort runs that spilled to disk this session.	PRINT 'Sort Spills: ' + @@SORT_SPILLS;

Example: Flow control based on row count

```
UPDATE target.Sales SET Status = 'Processed' WHERE Status = 'Pending';
IF @@ROWCOUNT > 0
BEGIN
    PRINT 'Updated ' + @@ROWCOUNT + ' records.';
END
```

1.5 INPUT and OUTPUT Variables

Control how variables are passed to and from sub-scripts.

```
DECLARE @BatchId INT INPUT = 0;
```

```
DECLARE @ExitStatus STRING OUTPUT = 'Pending';
```

CLI usage:

```
ETL-SQL run my_script.etlsql --var @BatchId=42 --var @Env=PROD
```

Script orchestration:

```
-- Parent
DECLARE @SubResult STRING;
RUN SCRIPT 'sub.etlsql' WITH (@Mode = 'FULL', @SubResult = @SubResult);
PRINT 'Sub finished: ' + @SubResult;

-- sub.etlsql
DECLARE @Mode      STRING INPUT;
DECLARE @SubResult STRING OUTPUT = 'OK';
```

1.6 USE PASSWORD

Sets the master decryption password for the session, used to decrypt **ENC**: connection strings.

```
USE PASSWORD = 'myMasterSecret';
CREATE CONNECTION db ON MSSQL('ENC:U2FsdGVkX1+...');
```

1.7 CLEAR SESSION

Deletes all temporary files, recovery manifests, and encrypted session state.

```
CLEAR SESSION;
```

1.8 Environment Sets (CREATE SETS / USE SETS / DROP SETS)

Named groups of variable assignments for switching between environments.

```
CREATE SETS !DEV
BEGIN
    @server    = 'dev-db.internal',
    @database  = 'DevWarehouse'
END

CREATE SETS !PROD
BEGIN
```



```
@server      = 'prod-db.internal',
@database    = 'ProdWarehouse';
SET WITH_PROMPT ON;    -- prompts for confirmation in interactive mode
END

USE SETS !DEV;
DROP SETS IF EXISTS !STAGING;
```

1.9 REQUIRE VERSION

Halts the script if the engine version does not meet the minimum.

```
REQUIRE >= '0.5.0';    -- VERSION keyword is optional
```

Supported operators: =, >, >=.

1.10 Variable Introspection

Use these statements to inspect variables in the current session or scope.

Statement	Purpose
SHOW VARIABLES	Lists all variables (Global + Local) available in the current context.
SHOW LOCAL VARIABLES	Lists only the variables declared in the current block/script scope.

All SHOW commands support the INTO #temp clause.

```
SHOW VARIABLES;
SHOW LOCAL VARIABLES INTO #vars;
SELECT * FROM #vars WHERE DataType = 'JSON';
```

2. Engine Configuration

2.1 SET WHAT_IF

Dry-run mode. Side-effecting operations are logged but not executed.

```
SET WHAT_IF ON;
DELETE FROM prod_db.logs WHERE log_date < '2024-01-01';    -- logged only
SET WHAT_IF OFF;
```

Suppressed: INSERT, UPDATE, DELETE, MERGE, TRUNCATE, BULK INSERT, all file/directory operations, SEND EMAIL, Docker actions, DDL.

Allowed: SELECT, DECLARE, SET, IF/WHILE, PRINT, CREATE CONNECTION.

2.2 SET PROFILING

Enables high-resolution statement-level monitoring. See [Section 2.6](#) for details.

```
SET PROFILING ON;
RUN SCRIPT 'heavy_transform.etlsql';
SET PROFILING OFF;
SHOW PROFILE INTO #perf;
```

2.3 SET SHOW_PASSWORD

Controls whether `USE PASSWORD` echoes the value in output. Default: `OFF`.

```
SET SHOW_PASSWORD ON;
USE PASSWORD = 'visible_for_debugging';
SET SHOW_PASSWORD OFF;
```

2.4 Security Overrides

Only honored when the path is within an approved Safe Zone. All overrides produce an audit entry.

Command	Description
SET ALLOW_FILE_TYPE_ACCESS ON/OFF	Allow file extensions not in the standard whitelist
SET ALLOW_FILE_TYPE_ACCESS = '.ext'	Add a specific extension (e.g. '.bak') to the authorized session whitelist
SET ALLOW_FILE_OPERATIONS = n	Overrides the default runaway protection limit (100) for file operations
SET ALLOW_RECURSIVE_LAYERS = n	Overrides the default recursion limit (5) for directory operations
SET ALLOW_GREATER_THAN_n_FILE ON/OFF	[Legacy] Alias for enabling large file counts
SET ALLOW_RECURSIVE_GREATER_THAN_n_LAYERS ON/OFF	[Legacy] Alias for enabling deep recursion

2.5 Performance & Spilling Thresholds

Override `appsettings.json` defaults for the current session.

Command	Default	Description
SET JOIN_SPILL_THRESHOLD = n	100,000	Rows before a join spills to disk
SET WINDOW_SPILL_THRESHOLD = n	100,000	Rows before window functions spill to disk

Command	Default	Description
<code>SET TEMP_TABLE_SPILL_THRESHOLD = n</code>	1,000,000	Rows before a <code>#temp</code> table spills to disk
<code>SET EXTERNAL_HASH_PARTITIONS = n</code>	32	Partitions used when spilling joins/windows
<code>SET EXTERNAL_SORT_CHUNK_SIZE = n</code>	50,000	Rows per chunk during external sort
<code>SET BATCHSIZE = n</code>	10,000	Rows per batch in the engine pipeline
<code>SET MAX_LAST_RESULT_ROWS = n</code>	50,000	Rows in the interactive display buffer
<code>SET MAX_RECURSIVE_DEPTH = n</code>	10,000	Max call depth for recursive CTEs or procedures
<code>SET MAX_IN_MEMORY_BATCHES = n</code>	100	Batches held before automatic <code>#temp</code> spill
<code>SET FOREACH_PAGE_SIZE = n</code>	10,000	Items fetched per page when iterating large collections
<code>SET MAX_MESSAGES = n</code>	1,000	Log/print messages captured in the session buffer
<code>SET MAX_FILE_OPERATIONS = n</code>	100	Filesystem operations allowed per script
<code>SET MAX_GENERATE_ROWS = n</code>	10,000	Rows per <code>GENERATE</code> statement (prevents resource exhaustion)
<code>SET MAX_PARALLEL_DEGREE = n</code>	8	Max concurrent branches inside a <code>PARALLEL</code> block

2.6 Observability & Telemetry

ETL-SQL provides two layers of performance monitoring: **Telemetry** and **Profiling**.

Telemetry (`SET TELEMETRY ON/OFF`)

Telemetry is the "always-on" (by default) lightweight monitoring layer. It tracks cumulative session-wide counters and provides the data for system variables like `@@ROWCOUNT` and `@@TOTAL_SPILLED_BYTES`.

- **Primary Use:** Monitoring overall job progress and resource pressure.
- **Scope:** Session-wide totals and the "last statement" summary.
- **Impact:** Negligible overhead (uses atomic counters).

Profiling (`SET PROFILING ON/OFF`)

Profiling is a high-resolution, opt-in monitoring layer that captures detailed timing and resource usage for **every individual statement** executed.

- **Primary Use:** Debugging performance bottlenecks and identifying slow queries.
- **Scope:** Statement-level granularity.
- **Impact:** Low, but creates telemetry objects for every statement; should be disabled in ultra-high-throughput production loops if not needed.
- **Commands:** Enables `SHOW PROFILE` and visual execution tree rendering in the IDE.

Feature	Telemetry	Profiling
Default	ON	OFF
Granularity	Session / Last Stmt	Per Statement
<code>@@variables</code>	✓	✓
<code>SHOW PROFILE</code>	X	✓
Execution Tree	X	✓
<code>SET MAX_STRING_RESULT_SIZE = n</code>	5,242,880	Max byte length of a string expression result (5 MB)
<code>SET REGEX_MATCH_TIMEOUT = n</code>	1,000	Milliseconds before a regex match is aborted
<code>SET MAX_GROUPING_SETS = n</code>	100	Max <code>CUBE/GROUPING SETS</code> combinations before abort
<code>SET MAX_SESSION_SIZE = n</code>	524,288,000	Max session state in bytes before eviction (~500 MB)
<code>SET SPILL_ENCRYPTION = ON/OFF</code>	ON	AES-256 encryption on spill files
<code>SET SPILL_COMPRESSION = ON/OFF</code>	ON	Brotli compression on spill files
<code>'SET SPILL_FORMAT = 'AUTO'</code>	<code>'JSON'</code>	<code>'PARQUET'</code>
<code>SET PARALLEL_MAX_DEGREE = n</code>	8	Max concurrent branches inside a <code>PARALLEL</code> block
<code>SET REGEX_TIMEOUT = n</code>	1,000	Milliseconds before a regex match is aborted
<code>SET TELEMETRY = ON/OFF</code>	ON	Collect high-cost execution metrics

3. Connections

3.1 `CREATE CONNECTION`

Registers a named data source in the current session.

```
-- General form
CREATE CONNECTION <alias> ON <ConnectorType>(<connection-string>);

-- Named-parameter form
CREATE CONNECTION <alias> ON <ConnectorType>(<key> = <value> [, ...]
);

-- Suppress error if already exists
```

```
CREATE CONNECTION <alias> IF NOT EXISTS ON <ConnectorType>(...);

-- Update existing connection
CREATE OR ALTER CONNECTION <alias> ON <ConnectorType>(...);
```

SQL Database Connectors

```
-- SQL Server (Common options: HOST, DATABASE, USER, PASSWORD, TRUSTED_CONNECTION,
-- USE_SSL, TRUST_SERVER_CERTIFICATE, APPLICATION_INTENT, CONNECT_TIMEOUT,
-- POOL_SIZE)
CREATE CONNECTION prod ON MSSQL(
  HOST = 'sql-server.company.com',
  DATABASE = 'Warehouse',
  TRUSTED_CONNECTION = TRUE,
  APPLICATION_INTENT = READONLY
);

-- PostgreSQL (Common options: HOST, PORT, DATABASE, USER, PASSWORD, SSL_MODE,
-- POOLING)
CREATE CONNECTION pg ON POSTGRES(
  HOST = 'pg-server.company.com',
  DATABASE = 'analytics',
  USER = 'etl',
  PASSWORD = ENC:...,
  SSL_MODE = 'REQUIRE'
);

-- Oracle (Common options: HOST, PORT, SERVICE_NAME, TNS_NAME, USER, PASSWORD)
CREATE CONNECTION ora ON ORACLE(
  HOST = 'ora-server.company.com',
  SERVICE_NAME = 'ORCL',
  USER = 'etl',
  PASSWORD = ENC:...
);

-- ODBC (Common options: DSN, DRIVER, SERVER, DATABASE, UID, PWD)
CREATE CONNECTION legacy ON ODBC(DSN = 'MyLegacyDSN');
```

File Connectors

```
-- Flat file (Common: PATH, FORMAT, DELIMITER, HEADER, ENCODING, SKIP, COMPRESS,
-- ENCRYPT)
CREATE CONNECTION sales_csv ON FLATFILE(
  PATH = 'C:\Data\sales.csv',
  FORMAT = 'CSV',
  DELIMITER = ',',
  HEADER = ON,
  ENCODING = 'UTF8'
);
```

```
-- Parquet (Common: PATH, COMPRESSION)
CREATE CONNECTION facts ON PARQUET(
  PATH = 'C:\Data\facts.parquet',
  COMPRESSION = 'SNAPPY'
);

-- JSON / XML (Common: PATH, ROOT_PATH, ENCODING)
CREATE CONNECTION config ON JSON(PATH = 'C:\Data\config.json', ROOT_PATH =
'$.settings');
```

Transfer Connectors

```
-- SFTP (Common: HOST, PORT, USER, PASSWORD, KEYFILE, PASSPHRASE)
CREATE CONNECTION remote_sftp ON SFTP(
  HOST = 'sftp.company.com',
  USER = 'etl',
  KEYFILE = 'C:\Keys\id_rsa'
);

-- Azure Blob Storage (Common: ACCOUNT_NAME, ACCOUNT_KEY, CONTAINER)
CREATE CONNECTION blob ON AZUREBLOB(
  ACCOUNT_NAME = 'mystorageaccount',
  CONTAINER = 'mycontainer',
  ACCOUNT_KEY = ENC:...
);
```

API & Notification Connectors

```
-- REST API (Common: URL, METHOD, AUTH_TYPE, TOKEN, BODY, ROOT_PATH, PAG_TYPE,
PAG_LIMIT)
CREATE CONNECTION api ON REST(
  URL = 'https://api.company.com/v1',
  AUTH_TYPE = 'BEARER',
  TOKEN = 'tkn_123',
  ROOT_PATH = '$.items'
);

-- SMTP (Common: HOST, PORT, USER, PASSWORD, USE_SSL, DEFAULT_FROM)
CREATE CONNECTION mailer ON SMTP(
  HOST = 'smtp.company.com',
  PORT = 587,
  USER = 'alerts@company.com',
  PASSWORD = ENC:...,
  USE_SSL = TRUE
);
```

Testing Connectors

```
-- In-memory mock database – useful for testing without a real database
CREATE CONNECTION testdb ON MOCKDB();

-- Local directory connector – treats a folder as a queryable table
CREATE CONNECTION logs_dir ON DIRECTORY('C:\Logs\') WITH(RECURSIVE=TRUE);
```

Service Connectors [PROPOSED]

```
-- Report Portal
CREATE CONNECTION portal ON REPORTPORTAL(
  HOST = 'report-server.company.com',
  PORT = 5001,
  USER = 'admin',
  PASSWORD = ENC:...
);

-- Orchestrator
CREATE CONNECTION orch ON ORCHESTRATOR(
  HOST = 'orch-server.company.com',
  PORT = 5100,
  USER = 'admin',
  PASSWORD = ENC:...
);
```

3.2 ALTER CONNECTION

Modifies an existing connection. Use this to rotate passwords or update server addresses without dropping the connection.

```
ALTER CONNECTION prod ON MSSQL(
  PASSWORD = ENC:...
);

-- Rename or change target only
ALTER CONNECTION stage ON POSTGRES('prod-server-v2');
```

3.3 DROP CONNECTION

```
DROP CONNECTION prod;
DROP CONNECTION prod IF EXISTS;
```

3.4 Connection Introspection

Use these statements to inspect the current state of connections in the session.

Statement	Purpose
:---	:---
<code>`SHOW CONNECTIONS`</code>	Lists all active connections with their type and summary details.
<code>`SHOW CONNECTION <name> CONFIG`</code>	Lists all configuration options for a specific connection (passwords redacted).

All ``SHOW`` commands support the ``INTO #temp`` clause to redirect results for further processing.

```
```sql
-- List all connections
SHOW CONNECTIONS;

-- Inspect configuration for a specific source
SHOW CONNECTION my_db CONFIG;

-- Programmatically inspect and filter config
SHOW CONNECTION my_db CONFIG INTO #cfg;
SELECT * FROM #cfg WHERE Option = 'SERVER';
```



### ### 3.3 Credential Helpers

Form	Usage
:---	:---
`ENC:...`	AES-256 encrypted value generated by `ENCRYPT VALUE 'plaintext'`
`USE PASSWORD = '...'`	Sets the session decryption key to unlock all `ENC:` values
`USE PASSWORD PROMPT`	Prompts the user interactively; password never written to disk

---

## ## 4. Control Flow

### ### 4.1 `IF / ELSE IF / ELSE`

```
```sql
IF @amount > 1000
BEGIN
    INSERT INTO #high SELECT * FROM #sales WHERE amount > 1000;
END
ELSE IF @amount > 500
BEGIN
    INSERT INTO #mid SELECT * FROM #sales WHERE amount > 500;
END
ELSE
BEGIN
    INSERT INTO #low SELECT * FROM #sales;
END
```

4.2 WHILE

```
DECLARE @i INT = 0;
WHILE @i < 10
BEGIN
    SET @i = @i + 1;
    IF @i = 5 CONTINUE;
    IF @i = 8 BREAK;
    PRINT @i;
END
```

4.3 FOR — Numeric Range

```
FOR @idx = 1 TO 10
BEGIN
    INSERT INTO #results (val) VALUES (@idx);
END
```

```
FOR @idx = 100 TO 95 STEP -1
BEGIN
    PRINT @idx;
END
```

4.4 FOR @row IN — Query Row Iteration

Executes the body once per row returned by the query. @row exposes columns via dot notation.

```
FOR @row IN (SELECT id, name, amount FROM sales_db.Orders WHERE status = 'Open')
BEGIN
    PRINT 'Order ' + @row.id + ': ' + @row.name;
    INSERT INTO #summary (OrderId, Name) VALUES (@row.id, @row.name);
END
```

For large result sets, consider using SET FOREACH_PAGE_SIZE to control how many rows are loaded at once.

4.5 FOREACH — Collection Iteration

Iterates through each item in a LIST variable, JSON array, or the rows of a #temp table.

- **Streaming Support:** When used with a subquery or a connection name, FOREACH uses a streaming cursor to minimize memory usage.
- **Auto-Unwrapping:** If the source is a table with a single column named Value, the iterator variable will contain the scalar value directly. Otherwise, it contains a ROW object.

```
-- Iterating a list
DECLARE @months LIST = ('Jan', 'Feb', 'Mar', 'Apr');
FOREACH @month IN @months
BEGIN
    PRINT 'Processing: ' + @month;
END

-- Iterating a #temp table (Dot notation for columns)
SELECT id, name INTO #users FROM src.users;
FOREACH @user IN #users
BEGIN
    PRINT 'User: ' + @user.id + ' - ' + @user.name;
END

-- Commonly used with FILE_LIST
DECLARE @files = FILE_LIST('C:\Data\Drops\', '*.csv');
FOREACH @file IN @files
BEGIN
    PRINT 'Loading: ' + @file.NAME;
```

```
BULK INSERT staging.daily FROM @file.PATH WITH (FORMAT = 'CSV', FIRSTROW = 2);
END
```

4.6 BREAK / CONTINUE / RETURN

Statement	Effect
BREAK	Exit the innermost WHILE / FOR / FOREACH loop
CONTINUE	Skip the rest of the current loop iteration
RETURN	Exit the current script or procedure immediately
RETURN <expr>	Exit and return a value to the caller

4.7 PRINT

PRINT supports two forms: statement and function.

Statement form — writes one or more expressions to the message log:

```
PRINT 'Starting nightly load...';
PRINT 'Processed: ' + @count + ' rows', TRUE;           -- with timestamp
PRINT GETDATE(), TRUE, 'yyyy-MM-dd HH:mm:ss';         -- formatted date

-- Multiple arguments (comma-separated)
PRINT 'User:', @Username, 'Status:', @Status;
```

Function form — **PRINT(expression)** — evaluates the expression and emits the result, useful inside compound expressions or when the statement form's comma-separation would be ambiguous:

```
PRINT(@@ROWCOUNT);
PRINT('Rows: ' + @count);

-- Inside a block where a single expression is expected
IF @debug = 1 PRINT(@msg);
```

Both forms accept the same optional **TRUE** timestamp flag and format string as the second and third arguments:

```
PRINT(@msg, TRUE, 'HH:mm:ss');
```

[!IMPORTANT] **PRINT** automatically masks any variable or expression result that is of type **SENSITIVE**, **SECRET**, or was declared with **PASSWORD**.

4.8 TRY...CATCH

```

BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO target_db.sales SELECT * FROM #staging;
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT 'Error: ' + ERROR_MESSAGE();
    THROW;    -- re-escalate to caller
END CATCH

```

4.9 RAISERROR / THROW

```

RAISERROR('Validation failed: missing required column', 16, 1);

THROW 50001, 'Batch ID not found in control table', 1;

```

4.10 WAITFOR / WAIT UNTIL

Pauses script execution. This is an ETL-SQL extension and is not standard SQL.

Syntax	Behavior
<code>WAITFOR DELAY 'hh:mm:ss[.fff]'</code>	Pause for a fixed duration
<code>WAITFOR TIME 'hh:mm:ss'</code>	Pause until a specific clock time
<code>WAITFOR (condition)</code>	Poll every 200 ms until condition is truthy
<code>WAIT UNTIL condition</code>	Preferred ELT-SQL alias for <code>WAITFOR (condition)</code>

```

WAITFOR DELAY '00:00:05';
WAITFOR DELAY '00:00:00.500';
WAITFOR TIME '23:30:00';

-- Polling for external state
WAIT UNTIL EXISTS (SELECT 1 FROM incoming_queue WHERE status = 'READY');
WAIT UNTIL (SELECT COUNT(*) FROM #batch_done) = 1;

```

4.11 ASSERT

Throws an `ExecutionException` if the condition is `FALSE` or `NULL`.

- **Statement Only:** `ASSERT` is a statement, not an expression. It cannot be used inside an `IF` condition or assigned to a variable.
- **Error Handling:** Failed assertions trigger the `CATCH` block of a `TRY...CATCH` statement, allowing for graceful cleanup or custom logging.

```

ASSERT (SELECT COUNT(*) FROM #staging) > 0, 'Staging table must not be empty';
ASSERT @total_amount >= 0, 'Negative balances are not allowed';

-- Catching a failed assertion
BEGIN TRY
    ASSERT (SELECT COUNT(*) FROM #errors) = 0, 'Validation errors detected';
END TRY
BEGIN CATCH
    PRINT 'Validation Failed: ' + ERROR_MESSAGE();
    RETURN;
END CATCH

```

4.12 EXPECT SCHEMA

Validates that a table or connection has the expected column names and type families.

```

EXPECT SCHEMA #staging (
    CustomerId INT,
    Name       VARCHAR,
    Amount     DECIMAL(18,2)
);

-- Warn instead of halting
EXPECT SCHEMA #staging (
    CustomerId INT,
    Name       VARCHAR
) ON DRIFT WARN;

-- Also works against named connections
EXPECT SCHEMA myConnection (
    OrderId    INT,
    OrderDate  DATE,
    Total      DECIMAL
);

```

Type families:

Family	Matched types
Integer	INT, INTEGER, BIGINT, SMALLINT, TINYINT
Decimal	DECIMAL, NUMERIC, MONEY, FLOAT, REAL, DOUBLE
String	VARCHAR, NVARCHAR, CHAR, TEXT, CLOB, STRING
Date	DATE, DATETIME, DATETIME2, TIMESTAMP, DATETIMEOFFSET
Boolean	BIT, BOOLEAN, BOOL
Binary	VARBINARY, BINARY, BLOB, IMAGE

Family	Matched types
Range	MINMAX

5. Querying (SELECT)

5.1 Complete Clause Reference

Clauses must appear in this syntactic order:

```

SELECT [DISTINCT] [TOP n [PERCENT] [WITH TIES]]
    <columns>
[INTO <target>]
FROM <source> [AS alias]
[JOIN | LEFT JOIN | RIGHT JOIN | FULL JOIN | CROSS JOIN
 | LEFT SEMI JOIN | LEFT ANTI JOIN <table>
    [HASH | LOOP | MERGE]          -- join algorithm hint
    ON <condition>]
[CROSS APPLY | OUTER APPLY (<subquery>) <alias>]
[WHERE <condition>]
[GROUP BY <columns> | ROLLUP(<cols>) | CUBE(<cols>) | GROUPING SETS(<sets>)]
[HAVING <condition>]
[PIVOT (<agg> FOR <col> IN (<vals>)) AS <alias>]
[UNPIVOT (<val_col> FOR <name_col> IN (<cols>)) AS <alias>]
[ORDER BY <col> [ASC|DESC] [, ...]]
[OFFSET n ROWS]
[FETCH NEXT n ROWS ONLY]
[LIMIT n]
[FOR JSON AUTO | PATH | RAW [, ROOT('name')] [, INCLUDE_NULL_VALUES] [,
WITHOUT_ARRAY_WRAPPER]]
[FOR XML AUTO | PATH | RAW [, ROOT('name')] [, ELEMENTS]];

```

5.2 INTO — Write Result to Target

```

SELECT id, name, category
INTO #temp_staging
FROM sales_db.transactions
WHERE created_at >= '2026-01-01';

```

5.3 TOP / LIMIT / OFFSET FETCH

```

SELECT TOP 10 * FROM #sales ORDER BY amount DESC;
SELECT TOP 5 PERCENT WITH TIES * FROM #sales ORDER BY amount DESC;
SELECT * FROM #sales ORDER BY amount DESC LIMIT 10;
SELECT * FROM #sales
ORDER BY amount DESC

```

```
OFFSET 20 ROWS
FETCH NEXT 10 ROWS ONLY;
```

5.4 JOIN Types

Syntax	Returns
JOIN / INNER JOIN	Rows with matching keys in both tables
LEFT JOIN / LEFT OUTER JOIN	All left rows; NULLs for unmatched right
RIGHT JOIN / RIGHT OUTER JOIN	All right rows; NULLs for unmatched left
FULL JOIN / FULL OUTER JOIN	All rows from both sides; NULLs for gaps
CROSS JOIN	Cartesian product
LEFT SEMI JOIN	Left rows where a match exists on the right
LEFT ANTI JOIN	Left rows where no match exists on the right

5.4.1 Join Algorithms

ETL-SQL supports three join algorithms. While the engine automatically chooses the best one based on table size and statistics, you can provide a hint to override it.

Algorithm	Hint	Description
Nested Loop	LOOP	Standard algorithm; iterates the outer table and probes the inner. Best for small datasets or when the inner side is indexed.
Hash Join	HASH	Builds an in-memory hash table of the inner source. Best for large, unsorted datasets. Higher memory usage.
Merge Join	MERGE	Simultaneously scans two sorted sources. Fastest algorithm for large datasets that are already ordered by the join keys.

```
SELECT * FROM #large AS a
HASH JOIN #lookup AS b ON a.id = b.id;    -- force hash join for performance
```

5.5 CROSS APPLY / OUTER APPLY

```
SELECT o.OrderId, t.LineItem
FROM Orders AS o
CROSS APPLY (SELECT * FROM OrderLines WHERE OrderId = o.OrderId) AS t;

SELECT o.OrderId, t.LineItem
FROM Orders AS o
OUTER APPLY (SELECT TOP 1 * FROM OrderLines WHERE OrderId = o.OrderId) AS t;
```

5.6 Hierarchical Aggregation

```
SELECT Region, Product, SUM(Amount) AS Total
FROM #sales
GROUP BY ROLLUP(Right, Product);

SELECT Region, Product, SUM(Amount) AS Total
FROM #sales
GROUP BY CUBE(Right, Product);

SELECT Region, Product, SUM(Amount) AS Total
FROM #sales
GROUP BY GROUPING SETS((Right, Product), (Right), ());
```

5.7 PIVOT / UNPIVOT

```
SELECT category, [Q1], [Q2], [Q3], [Q4]
FROM (SELECT category, quarter, amount FROM #sales) AS src
PIVOT (SUM(amount) FOR quarter IN ([Q1], [Q2], [Q3], [Q4])) AS pvt;

SELECT category, quarter, amount
FROM #quarterly_sales
UNPIVOT (amount FOR quarter IN ([Q1], [Q2], [Q3], [Q4])) AS unpvt;
```

5.8 FOR JSON / FOR XML

```
SELECT id, name, amount FROM #sales
FOR JSON PATH, ROOT('Sales'), INCLUDE_NULL_VALUES;

SELECT id, name FROM #sales
FOR XML PATH, ROOT('Employees'), ELEMENTS;
```

5.9 Window Functions

Window functions compute a value across a set of rows related to the current row without collapsing them into a single group. They appear in the **SELECT** column list and require an **OVER** clause.

```
SELECT
    id,
    name,
    amount,
    region,

    -- Ranking
    ROW_NUMBER() OVER (PARTITION BY region ORDER BY amount DESC) AS row_num,
```



```
RANK()          OVER (PARTITION BY region ORDER BY amount DESC) AS rnk,
DENSE_RANK()    OVER (PARTITION BY region ORDER BY amount DESC) AS dense_rnk,
NTILE(4)        OVER (ORDER BY amount)                          AS quartile,

-- Offset
LAG(amount, 1, 0) OVER (PARTITION BY region ORDER BY created_at) AS
prev_amount,
LEAD(amount, 1, 0) OVER (PARTITION BY region ORDER BY created_at) AS
next_amount,
FIRST_VALUE(amount) OVER (PARTITION BY region ORDER BY created_at) AS
first_in_region,
LAST_VALUE(amount)  OVER (PARTITION BY region ORDER BY created_at
                        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING) AS last_in_region,

-- Aggregate over window
SUM(amount)      OVER (PARTITION BY region)                      AS region_total,
AVG(amount)      OVER (PARTITION BY region)                      AS region_avg,
COUNT(*)        OVER (PARTITION BY region)                      AS region_count,
SUM(amount)      OVER (ORDER BY created_at
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_total

FROM #sales;
```

Frame syntax:

Clause	Meaning
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	All rows from the partition start to the current row
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING	Current row and one row on either side
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	All rows in the partition

6. Common Table Expressions (CTE)

```
-- Standard CTE
WITH HighSales AS (
  SELECT category, SUM(price) AS Total
  FROM sales_db.transactions
  GROUP BY category
)
SELECT * FROM HighSales WHERE Total > 10000;

-- CTE with explicit column aliases
WITH HighSales (Cat, TotalPrice) AS (
  SELECT category, SUM(price)
  FROM sales_db.transactions
```

```
        GROUP BY category
    )
    SELECT Cat FROM HighSales WHERE TotalPrice > 10000;

-- Multiple CTEs
WITH
    CTE_A AS (SELECT ...),
    CTE_B AS (SELECT ... FROM CTE_A WHERE ...)
SELECT * FROM CTE_B;

-- Recursive CTE
WITH RECURSIVE Counter AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM Counter WHERE n < 10
)
SELECT n FROM Counter;
```

7. Set Operations

```
SELECT region FROM #east_sales
UNION
SELECT region FROM #west_sales;

SELECT id FROM #batch_a
UNION ALL
SELECT id FROM #batch_b;

SELECT id FROM #full_list
EXCEPT
SELECT id FROM #processed;

SELECT id FROM #active
INTERSECT
SELECT id FROM #eligible;
```

8. Logical Operators & Filter Predicates

```
WHERE amount >= 100 AND status <> 'Cancelled'

WHERE category IN ('Electronics', 'Apparel')
    OR status NOT IN @exclusionList

WHERE email LIKE '%@company.com'
    AND code LIKE 'US\_%' ESCAPE '\'

WHERE EXISTS (SELECT 1 FROM #approved WHERE id = t.id)
```

```
WHERE NOT EXISTS (SELECT 1 FROM #blocked WHERE id = t.id)

WHERE region IS NOT NULL
      AND notes IS NULL
```

8.1 Member Access (Dot Notation)

Access columns of a row variable, fields of a JSON object, or properties of a system object.

Resolution order: Row columns → JSON fields → C# reflection properties (case-insensitive).

Object	Member	Description
Row variable (FOR @row IN)	.columnName	Column value during row iteration — see §4.4
FILE_LIST / REMOTE_FILE_LIST rows	.NAME, .PATH, .SIZE, etc.	File metadata columns — see §15.6
MINMAX variable	.MIN, .MAX	Range bounds — see §1.2
Docker alias	.CONNECTION_STRING	Host-mapped connection string — see §17
JSON variable	.fieldName	Dynamic field extraction — see §1.2

9. DML (Data Manipulation Language)

9.1 INSERT INTO

```
INSERT INTO sales_db.archive (category, TotalSales)
OUTPUT INSERTED.category, INSERTED.TotalSales INTO #AuditLog
SELECT category, SUM(amount) FROM #daily WHERE processed = 1
GROUP BY category;
```

9.2 UPDATE

```
UPDATE sales_db.archive
SET status = 'Closed', closed_at = GETDATE()
OUTPUT DELETED.status AS OldStatus, INSERTED.status AS NewStatus INTO #ChangeLog
WHERE created_at < '2020-01-01';
```

9.3 DELETE

```
DELETE FROM staging.temp_imports
OUTPUT DELETED.id INTO #deleted_ids
```

```
WHERE imported_at < DATEADD(DAY, -7, GETDATE());
```

9.4 MERGE (Upsert)

```
MERGE INTO target_db.Customers AS T
USING (SELECT * FROM #staging) AS S
ON T.UUID = S.UUID
WHEN MATCHED AND S.Checksum <> T.Checksum THEN
    UPDATE SET T.Name = S.Name, T.UpdatedAt = GETDATE()
WHEN NOT MATCHED BY TARGET THEN
    INSERT (UUID, Name) VALUES (S.UUID, S.Name)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

9.5 BULK INSERT

```
BULK INSERT target_db.DailyLogs (Name, Region, Amount)
FROM 'C:\Incoming\logs.csv'
WITH (
    FORMAT           = 'CSV',
    FIRSTROW         = 2,
    BATCHSIZE        = 10000,
    MAXERRORS        = 5,
    FIELDTERMINATOR  = ',',
    ROWTERMINATOR    = '\n',
    DATE_FORMAT      = 'yyyy-MM-dd',
    STRICT_SCHEMA    = ON
);
```

Supported formats: [CSV](#), [PARQUET](#), [AVRO](#), [EXCEL](#).

9.6 GENERATE

Produces synthetic data for testing.

```
GENERATE 100 ROWS INTO #test_data
WITH (SEED = 42)
AS (
    id          = 'SEQUENCE(1, 1)',
    category    = 'RANDOM(Electronics, Apparel, Home)',
    price       = 'RANDOM_DECIMAL(10, 500)',
    created     = 'SEQUENCE(2026-01-01, 1, DAY)'
);
```

Rule functions: `SEQUENCE(start, step [, unit])`, `RANDOM(val1, val2, ...)`, `RANDOM_INT(min, max)`, `RANDOM_DECIMAL(min, max)`.

9.7 TRUNCATE TABLE

```
TRUNCATE TABLE staging.Daily_Import;
```

10. DDL (Data Definition Language)

10.1 CREATE TABLE

```
CREATE TABLE #OrderItems (  
    OrderId      INT          IDENTITY PRIMARY KEY,  
    LineItem     INT          NOT NULL,  
    Amount       DECIMAL(18,2) NOT NULL CHECK(Amount >= 0),  
    Status       VARCHAR(20)  DEFAULT 'Pending',  
    CustomerId   INT          REFERENCES Customers(Id),  
    CONSTRAINT UQ_Line UNIQUE (OrderId, LineItem)  
);
```

[!NOTE] `CREATE OR ALTER` is not supported for tables. Use `DROP TABLE IF EXISTS` followed by `CREATE TABLE`, or use the `IF NOT EXISTS` clause.

10.2 ALTER TABLE

```
ALTER TABLE #staging ADD BatchId INT;  
ALTER TABLE #staging DROP COLUMN TempFlag;  
ALTER TABLE #staging RENAME COLUMN Name TO FullName;
```

-- Drop when done (auto-dropped at session end anyway) `DROP TABLE IF EXISTS #staging;`

10.3 DROP TABLE

```
DROP TABLE IF EXISTS #temp_staging;
```

10.4 CREATE INDEX / DROP INDEX

```
CREATE UNIQUE INDEX IX_Customers_Email ON Customers (Email ASC);  
DROP INDEX Customers.IX_Customers_Email;  
DROP INDEX IF EXISTS Customers.IX_Customers_Email;
```

[!NOTE] **CREATE OR ALTER** is not supported for indexes.

11. Execution Blocks

11.1 EXEC / EXECUTE — Execution & Pushdown

EXEC and **EXECUTE** are functional synonyms in ETL-SQL. They are used for executing dynamic SQL strings, stored procedures, or pushing native SQL blocks directly to a remote connection.

Native SQL Pushdown

Pushes a SQL block to a remote connection in its native dialect.

```
DECLARE @minId INT = 100;
DECLARE @status VARCHAR(20) = 'Active';

EXECUTE m_db INTO #results WITH (@minId, @status)
BEGIN
    SELECT t.id, t.name
    FROM dbo.Employee AS t
    WHERE t.id > ?1 AND t.status = ?2;
END
```

Parameters: ? = sequential, ?1/?2 = indexed.

Dynamic SQL

Executes a string as SQL. If **AT** is specified, it executes on the remote connection; otherwise, it executes locally as an ETL-SQL script.

```
-- Execute a string as an ETL-SQL script (local)
DECLARE @sql = 'SELECT * FROM #staging';
EXEC (@sql) INTO #results;

-- Push a dynamic string to a remote connection
EXECUTE ('SELECT TOP 10 * FROM Users ORDER BY LastLogin DESC') AT mssql_conn INTO
#top_users;
```

Stored Procedure Call

```
DECLARE @Count INT;
EXECUTE prod_db.dbo.sp_GetCustomerCount @Status = 'Active', @Count = @Count
OUTPUT;
```

```
-- Shorthand
EXEC ArchiveSales '2025-01-01';
```

Service Admin Blocks

Sends a block of admin statements to a **REPORTPORTAL** or **ORCHESTRATOR** connection.

```
-- Using EXECUTE
EXECUTE portal BEGIN
    CREATE USER 'john.doe' WITH (EMAIL = 'john@company.com', ROLE = Viewer);
    GRANT READ ON FOLDER '/Finance' TO GROUP 'Finance';
END

-- Using EXEC (Shorthand)
EXEC orch BEGIN
    CREATE JOB 'NightlyArchive' ON SCHEDULE EVERY 1 DAY AT '02:00' AS
        RUN SCRIPT '/scripts/nightly.etlsql';
END
```

Error behavior: Stop-on-first-error within each block. The block is not transactional — a failure mid-block leaves prior statements applied. **GO** as a sub-batch separator is a planned v1.1 extension.

11.3 PARALLEL

```
PARALLEL
BEGIN
    SELECT * INTO #Dim_Date FROM src.DateDim;
    SELECT * INTO #Dim_Product FROM src.ProductDim;
    SELECT * INTO #Dim_Region FROM src.RegionDim;
END
PRINT 'All dimensions loaded.';

-- With concurrency cap
PARALLEL(4)
BEGIN
    RUN SCRIPT 'load_region_north.etlsql';
    RUN SCRIPT 'load_region_south.etlsql';
    RUN SCRIPT 'load_region_east.etlsql';
    RUN SCRIPT 'load_region_west.etlsql';
    RUN SCRIPT 'load_region_central.etlsql';
END
```

11.4 RUN SCRIPT

```
RUN SCRIPT 'sub_process.etlsql' WITH (@batchId = 1234, @env = 'PROD');
```

12. Procedures & Functions

12.1 CREATE PROCEDURE

```
CREATE PROCEDURE ArchiveSales @olderThan DATE
AS
BEGIN
    INSERT INTO archive.sales SELECT * FROM prod.sales WHERE created_at <
@olderThan;
    DELETE FROM prod.sales WHERE created_at < @olderThan;
END;

EXEC ArchiveSales '2025-01-01';
```

12.2 CREATE FUNCTION

```
CREATE FUNCTION CalculateTax(@amount DECIMAL) RETURNS DECIMAL
AS
BEGIN
    RETURN @amount * 0.15;
END;

SELECT id, CalculateTax(price) AS Tax FROM #sales;
```

12.3 CREATE OR ALTER / DROP

```
CREATE OR ALTER PROCEDURE ArchiveSales @olderThan DATE AS BEGIN ... END;
CREATE OR ALTER FUNCTION CalculateTax(@amount DECIMAL) RETURNS DECIMAL AS BEGIN
... END;

DROP FUNCTION IF EXISTS CalculateTax;
DROP PROCEDURE IF EXISTS ArchiveSales;
```

13. Transactions

```
BEGIN TRANSACTION;    -- or BEGIN TRAN

-- ... operations ...

IF @@TRANCOUNT > 0
    COMMIT;            -- or COMMIT TRAN
```



```
-- On failure in CATCH:
ROLLBACK;          -- or ROLLBACK TRAN
```

14. Job Scheduling

14.1 CREATE JOB — Local Orchestrator

Registers a job with the local Orchestrator service.

```
-- EVERY interval syntax
CREATE JOB CleanupJob ON SCHEDULE EVERY 30 MINUTES AS
  RUN SCRIPT 'scripts/cleanup.etlsql';

CREATE JOB NightlyArchive ON SCHEDULE EVERY 1 DAY AT '02:00' AS
BEGIN
  INSERT INTO archive SELECT * FROM prod.logs WHERE log_date <
DATEADD(DAY,-30,GETDATE());
  DELETE FROM prod.logs WHERE log_date < DATEADD(DAY,-30,GETDATE());
END;

-- Cron syntax (equivalent; both forms are valid)
CREATE JOB WeeklyReport WITH (SCHEDULE = '0 8 * * MON') AS
  RUN SCRIPT 'scripts/weekly_report.etlsql';
```

Schedule intervals (EVERY form): SECONDS, MINUTES, HOURS, DAYS

Cron syntax: standard 5-field cron expression in the WITH (SCHEDULE = '...') form.

14.2 CREATE JOB — Remote Orchestrator [PROPOSED]

Targets a specific remote Orchestrator using AT <alias>. The alias must be a connection created with ON ORCHESTRATOR(...).

```
CREATE JOB 'NightlyArchive' AT orch ON SCHEDULE EVERY 1 DAY AT '02:00' AS
  RUN SCRIPT '/scripts/nightly_archive.etlsql';

CREATE JOB 'WeeklyReport' AT orch WITH (SCHEDULE = '0 8 * * MON') AS
  RUN SCRIPT '/scripts/weekly_report.etlsql';
```

[!NOTE] CREATE OR ALTER is not supported for jobs. Use DROP JOB and then CREATE JOB or use ALTER JOB to modify schedule/properties.

14.3 Job Management

```
-- Local
ALTER JOB CleanupJob SET SCHEDULE = EVERY 1 HOUR;
```

```

ALTER JOB WeeklyReport SET SCHEDULE = '0 9 * * MON';    -- cron form
ENABLE JOB CleanupJob;
DISABLE JOB CleanupJob;
DROP JOB IF EXISTS CleanupJob;
TRIGGER JOB CleanupJob;          -- manual one-off run (does not affect next
scheduled run)
KILL JOB <HistoryId>;            -- cancel a running instance by its history ID

SHOW JOBS;
SHOW JOB HISTORY;
SHOW JOB HISTORY NightlyArchive;
SHOW ACTIVE JOBS;                -- running instances only

SHOW JOBS INTO #jobs;
SHOW JOB HISTORY INTO #history;

```

15. File Operations

All paths are validated against the active Safe Zones before any I/O occurs. See [SET ALLOW_FILE_TYPE_ACCESS](#) and related overrides in §2.4.

15.1 File Statements

```

COPY FILE      '<source>' TO '<destination>' [WITH (OVERWRITE = ON|OFF)];
MOVE FILE      '<source>' TO '<destination>' [WITH (OVERWRITE = ON|OFF)];
RENAME FILE    '<source>' TO '<new_name>'    [WITH (OVERWRITE = ON|OFF)];
DELETE FILE    '<path>';
DELETE FILE    '<path>' IF EXISTS;

-- Wildcard sources
COPY FILE 'C:\Incoming\*.csv' TO 'C:\Archive\';

```

15.2 File Encryption / Compression

```

COMPRESS FILE '<source>' TO '<destination>' [WITH (OVERWRITE = ON|OFF)];
ENCRYPT FILE  '<source>' TO '<destination>' PASSWORD '<pwd>' [WITH (OVERWRITE =
ON|OFF)];
DECRYPT FILE  '<source>' TO '<destination>' PASSWORD '<pwd>' [WITH (OVERWRITE =
ON|OFF)];

```

15.3 Directory Statements

```

CREATE DIRECTORY '<path>' [IF NOT EXISTS];

COPY DIRECTORY  '<src>' TO '<dest>'          [WITH (OVERWRITE = ON|OFF)];

```

```
MOVE DIRECTORY    '<src>' TO '<dest>'    [WITH (OVERWRITE = ON|OFF)];
RENAME DIRECTORY  '<src>' TO '<new_name>' [WITH (OVERWRITE = ON|OFF)];

DELETE DIRECTORY      '<path>' [IF EXISTS];
DELETE DIRECTORY_CONTENTS '<path>' [WITH (RECURSIVE = ON|OFF)];

COMPRESS DIRECTORY '<src>' TO '<dest.zip>' [WITH (OVERWRITE = ON|OFF)];
ENCRYPT DIRECTORY   '<src>' TO '<dest>' PASSWORD '<pwd>' [WITH (OVERWRITE = ON|OFF,
RECURSIVE = ON|OFF)];
DECRYPT DIRECTORY   '<src>' TO '<dest>' PASSWORD '<pwd>' [WITH (OVERWRITE = ON|OFF,
RECURSIVE = ON|OFF)];
```

15.4 File Function Aliases

Underscore-style function aliases are available for backward compatibility:

```
COPY_FILE('src', 'dest' [, ON|OFF])
MOVE_FILE('src', 'dest' [, ON|OFF])
RENAME_FILE('src', 'new_name' [, ON|OFF])
DELETE_FILE('path')
COMPRESS_FILE('src', 'dest' [, ON|OFF])
ENCRYPT_FILE('src', 'dest', 'pwd' [, ON|OFF])
DECRYPT_FILE('src', 'dest', 'pwd' [, ON|OFF])
CREATE_DIRECTORY('path' [, ON|OFF])
DELETE_DIRECTORY('path')
DELETE_DIRECTORY_CONTENTS('path' [, RECURSIVE = ON|OFF])
```

15.5 Remote File Transfer

```
-- Upload to remote
SEND FILE '<local_path>' TO '<remote_path>' AT <connection> [WITH (OVERWRITE =
ON|OFF)];

-- Download from remote
RECEIVE FILE FROM '<remote_path>' TO '<local_path>' AT <connection> [WITH
(OVERWRITE = ON|OFF)];
```

15.6 Filesystem Query Functions

Function	Returns
FILE_EXISTS(path)	TRUE if the file exists
DIRECTORY_EXISTS(path)	TRUE if the directory exists
FILE_LIST(path [, pattern [, recursive]])	Table: Name, Path, Extension, Size, LastModified

Function	Returns
REMOTE_FILE_LIST(conn, path)	Table: Name, FullPath, Size, LastModified, IsDirectory
<pre>IF FILE_EXISTS('C:\Incoming\payload.csv') COPY FILE 'C:\Incoming\payload.csv' TO 'C:\Archive\'; -- Query files as a table SELECT Name, Size, LastModified INTO #new_files FROM FILE_LIST('C:\Incoming\', '*.csv', TRUE) WHERE LastModified >= DATEADD(HOUR, -24, GETDATE()) ORDER BY LastModified DESC; -- Remote directory listing SELECT Name, Size, LastModified INTO #remote_files FROM REMOTE_FILE_LIST(remote_sftp, '/var/ftp/incoming/') WHERE LastModified >= DATEADD(HOUR, -24, GETDATE());</pre>	

16. Email

Syntax note: `SEND EMAIL` is the canonical form. `EMAIL SEND` is a supported alias. `AT <connection>` is the standard keyword for specifying the SMTP connection, consistent with `SEND FILE ... AT conn`, `RECEIVE FILE ... AT conn`, and `CREATE JOB ... AT orch`.

16.1 SEND EMAIL

<pre>SEND EMAIL TO 'recipient@company.com' FROM 'sender@company.com' -- omit to use DEFAULT_FROM on the SMTP connection SUBJECT 'Pipeline Status' BODY 'The nightly load completed successfully.' [CC 'manager@company.com'] [BCC 'audit@company.com'] [ATTACH 'C:\Reports\summary.pdf'] [ATTACH 'C:\Reports\detail.csv'] AT mailer;</pre>	
---	--

Clause	Required	Notes
TO	Yes	One or more addresses
FROM	No	Defaults to <code>DEFAULT_FROM</code> on the SMTP connector

Clause	Required	Notes
SUBJECT	Yes	
BODY	Yes	Plain text or HTML
CC	No	
BCC	No	
ATTACH	No	Local file path; repeatable
AT	No	Defaults to the last configured SMTP connection

16.2 Example

```

CREATE CONNECTION mailer ON SMTP(
  HOST      = 'smtp.company.com',
  PORT      = 587,
  USER      = 'alerts@company.com',
  PASSWORD  = ENC:...,
  USE_SSL   = true,
  DEFAULT_FROM = 'alerts@company.com'
);

BEGIN TRY
  -- ... pipeline work ...
  SEND EMAIL
    TO      'ops@company.com'
    SUBJECT 'Nightly Load - SUCCESS'
    BODY    'All ' + @rowCount + ' rows loaded.'
    AT      mailer;
END TRY
BEGIN CATCH
  SEND EMAIL
    TO      'ops@company.com'
    SUBJECT 'Nightly Load - FAILED'
    BODY    'Error at step ' + @step + ': ' + ERROR_MESSAGE()
    ATTACH  'C:\Logs\nightly_error.log'
    AT      mailer;
  THROW;
END CATCH

```

17. Containerized Test Databases (USE DOCKER)

17.1 Spawning a Container

```

USE DOCKER('<image>') [AS <alias>];

USE DOCKER('mcr.microsoft.com/mssql/server:2022-latest') AS mssql_db;

```

```
USE DOCKER('postgres:15-alpine') AS pg_db;
USE DOCKER('gvenzl/oracle-free:latest') AS ora_db;
```

After startup the connection string is available via the alias:

```
DECLARE @conn VARCHAR(500) = mssql_db.CONNECTION_STRING;
CREATE CONNECTION stage_db ON MSSQL(@conn);
```

17.2 Supported Images

Database	Image pattern	Default credentials	Port
SQL Server	contains <code>mssql</code>	<code>sa / Password123!</code>	1433
PostgreSQL	contains <code>postgres</code>	<code>postgres / postgres</code>	5432
Oracle	contains <code>oracle</code>	<code>system / oracle</code>	1521

17.3 Lifecycle Commands

Command	Effect
<code>START DOCKER <alias></code>	Resume a stopped container
<code>STOP DOCKER <alias></code>	Stop the container (state preserved)
<code>PAUSE DOCKER <alias></code>	Suspend CPU (faster resume than stop/start)
<code>CLOSE DOCKER <alias></code>	Destroy container and all its state
<code>CLOSE_DOCKER</code>	Destroy all containers in the session

Function-style aliases: `START_DOCKER`, `STOP_DOCKER`, `CLOSE_DOCKER`.

Containers are **not** automatically closed when a script ends. Always include an explicit `CLOSE_DOCKER` or wrap in `TRY...CATCH`.

17.4 Multiple Containers

```
USE DOCKER('mcr.microsoft.com/mssql/server:2022-latest') AS src;
USE DOCKER('postgres:15-alpine') AS dst;

CREATE CONNECTION source_db ON MSSQL(src.CONNECTION_STRING);
CREATE CONNECTION target_db ON POSTGRES(dst.CONNECTION_STRING);

SELECT * INTO #tmp FROM source_db.dbo.Customers;
INSERT INTO target_db.public.customers SELECT * FROM #tmp;

CLOSE_DOCKER;
```

18. Introspection & Diagnostics

18.1 Metadata

```
SHOW CONNECTIONS [INTO #temp];
SHOW TABLES [ON conn] [INTO #temp];
SHOW COLUMNS FOR conn.TableName [INTO #temp];
SHOW VERSION [INTO #temp];
SHOW SAFE ZONES [INTO #temp];
```

18.2 Session

```
SHOW VARIABLES [INTO #temp];
SHOW LOCAL VARIABLES [INTO #temp];
SHOW PROFILE [INTO #temp];
SHOW LINEAGE [FOR table] [INTO #temp];
SHOW TAGS FOR TABLE t [COLUMN c] [INTO #temp];
```

18.3 Jobs

```
SHOW JOBS [INTO #temp];
SHOW ACTIVE JOBS [INTO #temp];
SHOW JOB HISTORY [<jobName>] [INTO #temp];
KILL JOB <HistoryId>;
```

18.4 Analysis

```
EXPLAIN SELECT * FROM conn.Orders WHERE status = 'Open';
LINT 'scripts/nightly_load.etlsql';
```

18.5 Help

```
HELP CONNECTION MSSQL;           -- connector-specific options
HELP CONNECTION POSTGRES;
HELP VARIABLES;                  -- list all @@ system variables
HELP FUNCTIONS;                  -- list all built-in functions
HELP TARGET <name>;              -- documentation for a specific statement or keyword
HELP TYPE <name>;                -- documentation for a data type (e.g. HELP TYPE
MINMAX)
```

18.6 Script Metadata Header

```
/*
  @author:      Chuck
  @version:     1.2.3
  @description: Nightly cleanup of staging tables
*/
```

Supported tags: `@author`, `@version`, `@description`, or any custom `@key: value` pair. `@author` defaults to the current system user if omitted.

Appendix A: Report-SQL Grammar (`.rptsq1` files)

`.rptsq1` files are standard ETL-SQL scripts with the following additional statement types. For the full user guide see [Docs/Report_SQL_Guide.md](#).

A.1 SET REPORT

```
SET REPORT TITLE      = 'Monthly Sales Dashboard';
SET REPORT DESCRIPTION = 'Revenue by region and product line.';
```

A.2 CREATE DATASET

```
CREATE DATASET <name>
  [REFRESH EVERY '<interval>']
  [TTL = '<duration>']
  [COMPRESS = ON|OFF]
  [ENCRYPT = MACHINE | PASSWORD | KEYFILE]
  [PASSWORD = '<password>']
  [KEYFILE = '<path>']
AS ( SELECT ... );
```

Interval format: `<n>s`, `<n>m`, `<n>h`, `<n>d`.

A.3 CREATE VISUAL

```
CREATE VISUAL <name> AS <type> (
  [SOURCE      = &dataset | #table | ( SELECT ... ),]
  [TITLE       = '<string>',]
  [SUBTITLE    = '<string>',]
  [TOOLTIP     = '<string>',]
  [MAPPINGS    ( role = column [, ...] ),]
  [OPTIONS     ( key = value [, ...]
                [, X_AXIS (...)] [, Y_AXIS (...)]
```



```

        [, COLORS ( key = '#hex' [, ...] )]
        [, LEGEND ( position = top|bottom|left|right )]
        [, CROSS_FILTER = true] ),]
[STYLE      ( key = value [, ...] ),]
[SERIES      ( BAR|LINE column [, ...] ),]
[FORMATTING ( column op threshold THEN '<color>' [, ...] ),]
[OVERLAYS    ( <VisualName> [, ...] ),]
[ACTIONS     ( trigger = action [, ...] )]
);

```

Visual types: BAR, HBAR, LINE, SCATTER, BUBBLE, RADAR, PIE, DONUT, COMBO, BOXPLOT, TREEMAP, HEATMAP, GAUGE, FUNNEL, WATERFALL, CANDLESTICK, MAP, TABLE, CARD, TEXT, IMAGE, SLICER, DATEPICKER, SLIDER, MULTISELECT, SEARCH

FORMATTING operators: <, >, <=, >=, =, <>

Action forms:

```

ON_CLICK = DRILL_DOWN(Target = <VisualName>, Key = <column>)
ON_CLICK = NAVIGATE(<PageName>)
ON_CHANGE = SET_PARAMETER(@paramName, <columnRef>)
ON_CHANGE = REFRESH
ON_CLICK = REFRESH
ON_CLICK = RESET_PARAMETERS

```

A.3.1 Filter Visuals (SLICER, DATEPICKER, etc.)

Filter visuals differ from charts in that they typically bind to a parameter via **ON_CHANGE**.

```

CREATE VISUAL RegionFilter AS SLICER (
  SOURCE = (SELECT DISTINCT region FROM #summary),
  MAPPINGS (VALUE = region),
  ACTIONS (ON_CHANGE = SET_PARAMETER(@region, region))
);

CREATE VISUAL DateFilter AS DATEPICKER (
  OPTIONS (TYPE = 'RANGE', FORMAT = 'yyyy-MM-dd'),
  ACTIONS (ON_CHANGE = SET_PARAMETER(@date_range, value))
);

```

A.4 CREATE PAGE

```

CREATE PAGE <name> AS LAYOUT (
  STRUCTURE = '<css-grid-template-areas>',
  MAP (
    '<slot>' = <VisualOrContainerName>
    [, '<slot>' = <name> ...]
  )
);

```

```

)
[, STYLE ( key = value [, ...] )]
)
[WITH PARAMETERS ( @param [AS type] [DEFAULT default | = default] [, ...] )]
;

```

STRUCTURE uses CSS grid-template-areas: space-separated slot letters per row, rows separated by /. Example: 'A A / B C'.

A.5 CREATE CONTAINER

```

CREATE CONTAINER <name> AS BOX|SCROLL (
  [STYLE ( key = value [, ...] ),]
  VISUALS ( <VisualName> [, ...] )
);

```

A.6 CREATE NAVIGATION

```

CREATE NAVIGATION <name> AS TAB|BUTTON|LINK (
  [ORIENTATION = HORIZONTAL|VERTICAL,]
  [DEFAULT = <PageName>]
)
WITH PAGES ( <PageName> [, ...] );

```

A.7 CREATE STYLE

```

CREATE STYLE <name> (
  BACKGROUND-COLOR = '#1e1e2e',
  COLOR             = '#cdd6f4',
  FONT-SIZE         = '14px',
  FONT-WEIGHT       = 'bold',
  BORDER            = '1px solid #ccc',
  BORDER-RADIUS     = '8px',
  PADDING           = '12px',
  MARGIN            = '4px',
  HEIGHT            = '300px',
  WIDTH             = '100%',
  THEME             = light | dark
);

```

A.8 CREATE BUTTON

```

CREATE BUTTON <name> (
  TITLE = '<string>',

```

```
[STYLE = <StyleName> | ( key = value [, ...] ),]
ACTIONS ( trigger = action [, ...] )
);
```

A.9 ALTER / DROP / CREATE OR ALTER

All report object types support these forms:

```
ALTER VISUAL      <name> ( <clause> [, ...] );
ALTER PAGE       <name> ( <clause> [, ...] );
ALTER CONTAINER  <name> ( <clause> [, ...] );
ALTER BUTTON     <name> ( <clause> [, ...] );
ALTER STYLE      <name> ( <clause> [, ...] );
ALTER NAVIGATION <name> ( <clause> [, ...] );
ALTER DATASET    <name> ( <clause> [, ...] );

DROP VISUAL      [IF EXISTS] <name>;
DROP PAGE       [IF EXISTS] <name>;
DROP CONTAINER  [IF EXISTS] <name>;
DROP BUTTON     [IF EXISTS] <name>;
DROP STYLE      [IF EXISTS] <name>;
DROP NAVIGATION [IF EXISTS] <name>;
DROP DATASET    [IF EXISTS] <name>;

CREATE OR ALTER VISUAL      <name> AS <type> ( ... );
CREATE OR ALTER PAGE       <name> AS LAYOUT ( ... );
CREATE OR ALTER DATASET    &<name> ... AS ( SELECT ... );
CREATE OR ALTER STYLE      <name> ( ... );
CREATE OR ALTER BUTTON     <name> ( ... );
CREATE OR ALTER CONTAINER  <name> AS BOX|SCROLL ( ... );
CREATE OR ALTER NAVIGATION <name> AS TAB|BUTTON|LINK ( ... ) WITH PAGES ( ... );
```

Appendix B: Report Portal Admin Language [PROPOSED]

Portal admin statements execute inside an **EXECUTE portal BEGIN...END** block. The **portal** alias must be a connection created with **ON REPORTPORTAL(...)**.

```
CREATE CONNECTION portal ON REPORTPORTAL(
  HOST = 'report-server.company.com',
  PORT = 5001,
  USER = 'admin',
  PASSWORD = ENC:...
);

EXECUTE portal BEGIN

-- =====
-- USERS
```

```

-- =====
CREATE USER 'john.doe'
    WITH (EMAIL = 'john@company.com', PASSWORD = ENC:..., ROLE = Viewer);

ALTER USER 'john.doe' SET EMAIL      = 'john.doe@newdomain.com';
ALTER USER 'john.doe' SET ROLE      = Publisher;
ALTER USER 'john.doe' SET PASSWORD  = ENC:...;
ALTER USER 'john.doe' ENABLE;
ALTER USER 'john.doe' DISABLE;

DROP USER 'john.doe';
DROP USER 'john.doe' CASCADE;  -- also removes subscriptions, sessions, group
memberships

SHOW USERS          [INTO #users];
SHOW USER 'john.doe' [INTO #detail];

-- =====
-- GROUPS
-- =====
CREATE GROUP 'Finance' WITH (DESCRIPTION = 'Finance department');
ALTER GROUP 'Finance' SET DESCRIPTION = 'Finance and Accounting';
DROP GROUP 'Finance';
DROP GROUP 'Finance' CASCADE;  -- removes memberships and ACL entries;
users remain

ADD USER 'john.doe' TO GROUP 'Finance';
REMOVE USER 'john.doe' FROM GROUP 'Finance';

SHOW GROUPS          [INTO #groups];
SHOW GROUP 'Finance' [INTO #detail];  -- includes member list
SHOW GROUPS FOR USER 'john.doe' [INTO #groups];

-- =====
-- FOLDERS
-- Folders are catalog containers – distinct from filesystem directories.
-- =====
CREATE FOLDER '/Finance';
CREATE FOLDER '/Finance/Monthly';

ALTER FOLDER '/Finance/Monthly' SET NAME      = 'Monthly Reports';
ALTER FOLDER '/Finance/Monthly' SET PARENT = '/Shared';

DROP FOLDER '/Finance/Monthly Reports';
DROP FOLDER '/Finance' CASCADE;  -- removes all child folders and their
reports

SHOW FOLDERS          [INTO #folders];  -- full tree visible to
caller
SHOW FOLDERS UNDER '/Finance' [INTO #folders];  -- subtree only
SHOW FOLDER '/Finance' [INTO #detail];  -- detail + ACL entries +
report count

-- =====

```

```

-- PERMISSIONS
-- Permissions are granted to groups, not individual users.
-- Levels: READ < EXECUTE < MANAGE
-- =====
GRANT READ      ON FOLDER '/Finance'      TO GROUP 'Finance';
GRANT EXECUTE   ON FOLDER '/Finance'      TO GROUP 'FinanceAnalysts';
GRANT MANAGE    ON FOLDER '/Finance'      TO GROUP 'FinanceAdmins';
REVOKE READ     ON FOLDER '/Finance'      FROM GROUP 'Finance';

SHOW PERMISSIONS ON FOLDER '/Finance'     [INTO #perms];
SHOW PERMISSIONS FOR GROUP 'Finance'     [INTO #perms];

-- =====
-- REPORT CATALOG
-- PUBLISH points the portal at an existing .rptsql file.
-- =====
PUBLISH REPORT 'Monthly Sales'
  FROM '/reports/finance/monthly_sales.rptsql'
  IN FOLDER '/Finance'
  WITH (DESCRIPTION = 'Monthly revenue by region');

ALTER REPORT 'Monthly Sales' SET FOLDER      = '/Finance/Archive';
ALTER REPORT 'Monthly Sales' SET DESCRIPTION = 'Archived monthly revenue
report';
ALTER REPORT 'Monthly Sales' SET NAME        = 'Monthly Sales (Archive)';

DROP REPORT 'Monthly Sales';
DROP REPORT 'Monthly Sales' CASCADE;  -- also removes snapshots and
subscriptions

SHOW REPORTS                      [INTO #reports];
SHOW REPORTS IN FOLDER '/Finance' [INTO #reports];
SHOW REPORT 'Monthly Sales'      [INTO #detail];

-- =====
-- SNAPSHOTS
-- =====
DROP SNAPSHOT FOR REPORT 'Monthly Sales';  -- force rebuild on next view
REBUILD SNAPSHOT FOR REPORT 'Monthly Sales';  -- rebuild now in background

SHOW SNAPSHOTS                      [INTO #snaps];

-- =====
-- SUBSCRIPTIONS
-- Group membership is evaluated at delivery time, not creation time.
-- =====
CREATE SUBSCRIPTION FOR REPORT '/Finance/MonthlySales'
  DELIVER TO 'john.doe'
  SCHEDULE '0 8 * * * MON'
  FORMAT PDF
  AT smtp;

CREATE SUBSCRIPTION FOR REPORT '/Finance/MonthlySales'
  DELIVER TO GROUP 'Finance'

```

```

    ON REFRESH                                -- fires whenever the dataset refreshes
    FORMAT BOTH                               -- PDF and CSV
    AT smtp;

ALTER SUBSCRIPTION 5 SET SCHEDULE = '0 9 * * MON';
ALTER SUBSCRIPTION 5 ENABLE;
ALTER SUBSCRIPTION 5 DISABLE;
DROP SUBSCRIPTION 5;

SHOW SUBSCRIPTIONS                           [INTO #subs];
SHOW SUBSCRIPTIONS FOR REPORT '/Finance/MonthlySales' [INTO #subs];

-- =====
-- SESSION MANAGEMENT
-- =====
DISCONNECT USER 'dr.allen';                  -- force logout; invalidates active
session
REVOKE TOKENS FOR USER 'dr.allen';           -- invalidate all JWT refresh tokens

SHOW ACTIVE SESSIONS [INTO #sessions];

-- =====
-- SERVICE CONTROL
-- RESTART sends 202 Accepted then restarts. SHUTDOWN sends 202 then stops.
-- START is not available via script (service is not running to receive it).
-- =====
RESTART PORTAL;
SHUTDOWN PORTAL;

-- =====
-- PORTAL METADATA QUERIES
-- Use the connection alias as a schema prefix.
-- All standard SELECT clauses (WHERE, ORDER BY, JOIN, etc.) are supported.
-- =====
SELECT * FROM portal.Users;
SELECT * FROM portal.Groups;
SELECT * FROM portal.UserGroups;
SELECT * FROM portal.Folders;
SELECT * FROM portal.FolderAcl;
SELECT * FROM portal.Reports;
SELECT * FROM portal.ReportSnapshots;
SELECT * FROM portal.Subscriptions;
SELECT * FROM portal.ActiveSessions;
SELECT * FROM portal.AuditLog WHERE Action = 'LOGIN_FAILED' AND Timestamp >
DATEADD(DAY,-7,GETDATE());

-- Example: who has access to the Finance folder?
SELECT u.Username, g.Name AS GroupName, a.Permission
FROM portal.Users u
JOIN portal.UserGroups ug ON u.Id = ug.UserId
JOIN portal.Groups g      ON ug.GroupId = g.Id
JOIN portal.FolderAcl a   ON a.GroupId = g.Id
JOIN portal.Folders f     ON a.FolderId = f.Id
WHERE f.Path = '/Finance'
```

```
ORDER BY a.Permission DESC, u.Username;

END
```

Permission levels:

Level	Can do
READ	View the report exists; view cached snapshots and exports
EXECUTE	Run the report with custom parameters; trigger manual refresh
MANAGE	Edit metadata, schedules, ACL; delete reports and folders
Admin role	Implicit MANAGE everywhere + user/group administration
Publisher role	MANAGE in folders where granted

Appendix C: Orchestrator Remote Management [PROPOSED]

Orchestrator admin statements execute inside an `EXECUTE orch BEGIN...END` block. The `orch` alias must be a connection created with `ON ORCHESTRATOR(...)`.

For targeting a remote Orchestrator from a standalone `CREATE JOB` statement (outside a block), use the `AT <alias>` form documented in §14.2.

```
CREATE CONNECTION orch ON ORCHESTRATOR(
  HOST = 'orch-server.company.com',
  PORT = 5100,
  USER = 'admin',
  PASSWORD = ENC:...
);

EXECUTE orch BEGIN

  -- =====
  -- JOB MANAGEMENT
  -- Both EVERY and cron schedule forms are valid.
  -- =====
  CREATE JOB 'NightlyArchive' ON SCHEDULE EVERY 1 DAY AT '02:00' AS
    RUN SCRIPT '/scripts/nightly_archive.etlsql';

  CREATE JOB 'WeeklyReport' WITH (SCHEDULE = '0 8 * * MON') AS
    RUN SCRIPT '/scripts/weekly_report.etlsql';

  ALTER JOB 'NightlyArchive' SET SCHEDULE = EVERY 1 DAY AT '03:00';
  ALTER JOB 'WeeklyReport' SET SCHEDULE = '0 9 * * MON';

  ENABLE JOB 'NightlyArchive';
  DISABLE JOB 'NightlyArchive';
  DROP JOB 'NightlyArchive' IF EXISTS;
```

```

    TRIGGER JOB 'NightlyArchive';          -- manual one-off; does not affect next
scheduled run
    KILL     JOB <HistoryId>;              -- cancel a running instance by its
history ID

-- =====
-- ORCHESTRATOR METADATA QUERIES
-- =====

SHOW JOBS          [INTO #jobs];
SHOW ACTIVE JOBS   [INTO #active];
SHOW JOB HISTORY   [INTO #history];
SHOW JOB HISTORY 'NightlyArchive' [INTO #history];

SELECT * FROM orch.Jobs;
SELECT * FROM orch.JobHistory WHERE Status = 'FAILED' AND StartedAt >
DATEADD(DAY, -7, GETDATE());
SELECT * FROM orch.ActiveJobs;

-- Example: jobs that failed more than once in the last week
SELECT JobName, COUNT(*) AS FailCount
FROM orch.JobHistory
WHERE Status = 'FAILED'
      AND StartedAt > DATEADD(DAY, -7, GETDATE())
GROUP BY JobName
HAVING COUNT(*) > 1
ORDER BY FailCount DESC;

END

```